

Software Developer
Unità Formativa (UF): Containers, Microservizi -
Serverless
Docente: Denis Maggiorotto
Titolo argomento: Docker Compose

Docker Compose



Cos'è Docker Compose

Docker Compose è uno strumento per definire ed eseguire applicazioni compose (multi-container) tramite Docker.

Le applicazioni multi-container vengono definite «as code» all'interno di uno o più file **docker-compose.yaml** per poi essere eseguite (o terminate) con un singolo comando.

Funzionalità

Tramite Docker Compose è possibile:

- Eseguire ambienti applicativi multipli su di un singolo host

- Preservare i dati in volumi quando i container vengono creati

- Ri-creare solo i container che sono cambiati

- Definire variabili che possono poi essere richiamate nel `docker-compose.yml` file

- Suddividere le dichiarazioni in file multipli

Verifica versione

Con Docker for Desktop, l'utility Docker Compose è già installata

Su Linux invece:

```
$ sudo curl -L  
"https://github.com/docker/compose/releases/download  
/1.25.5/docker-compose-$(uname -s)-$(uname -m)" -o  
/usr/local/bin/docker-compose
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

Verifica

```
$ docker-compose -version  
docker-compose version 1.25.4, build 8d51620a
```

Prima di Docker Compose

```
$ docker build -t myapp .  
$ docker network create frontend  
$ docker network create backend  
$ docker run -d --network backend redis:alpine  
$ docker run -d -p 5000:5000 --network frontend --name my-application myapp  
$ docker network connect backend my-application
```

Con Docker Compose

```
$ docker-compose up
```

Nel file docker-compose.yaml

Version

Services

-> Build

-> Image

-> Environment

-> Ports

-> Volumes

Volumes

Networks

Esempio di docker-compose.yml

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
  redis:
    image: "redis:alpine"
    networks:
      - backend
networks:
  frontend:
  backend:
```

Comandi utili

Avvio (aggiungere `-d` per background mode)

```
$ docker-compose up
```

Verifica container in esecuzione

```
$ docker-compose ps
```

Verifica dei logs

```
$ docker-compose logs
```

Stop dei container e rimozione

```
$ docker-compose down
```

Tutti i comandi docker-compose

build	Build or rebuild services
help	Get help on a command
kill	Kill containers
logs	View output from containers
port	Print the public port for a port binding
ps	List containers
pull	Pulls service images
rm	Remove stopped containers
run	Run a one-off command
scale	Set number of containers for a service
start	Start services
stop	Stop services
restart	Restart services
up	Create and start containers

Build o image

E' possibile generare una nuova immagine

E' possibile referenziare un'immagine esistente

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
  redis:
    image: "redis:alpine"
    networks:
      - backend
networks:
  frontend:
  backend:
```

Depends_on

Determina le dipendenze tra container

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
  redis:
    image: "redis:alpine"
    networks:
      - backend
networks:
  frontend:
  backend:
```

Networks

Utilizzo delle networks

Creazione delle networks

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
  redis:
    image: "redis:alpine"
    networks:
      - backend
networks:
  frontend:
  backend:
```

Environment

E' possibile definire variabili d'ambiente

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
    restart: always
    environment:
      MYVAR: "myvalue"
    env_file: .env
  redis:
    image: "redis:alpine"
    networks:
      - backend
    restart: always
networks:
  frontend:
  backend:
```

Restart

Politica di restart:

- No
- Always
- On-failure
- Unless-stopped

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
    restart: always
    environment:
      MYVAR: "myvalue"
    env_file: .env
  redis:
    image: "redis:alpine"
    networks:
      - backend
    restart: always
networks:
  frontend:
  backend:
```

Volumes

Utilizzo dei volumi

Creazione dei volumi

```
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    networks:
      - frontend
      - backend
    depends_on:
      - redis
    restart: always
    environment:
      MYVAR: "myvalue"
    env_file: .env
  redis:
    image: "redis:alpine"
    networks:
      - backend
    restart: always
    volumes:
      - dbdata:/var/dbdata
networks:
  frontend:
  backend:
volumes:
  dbdata:
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 17 – Docker Compose

Assignment 04 - Multicontainer application with Docker Compose